

Assignment-21.2

Name: Saisharan

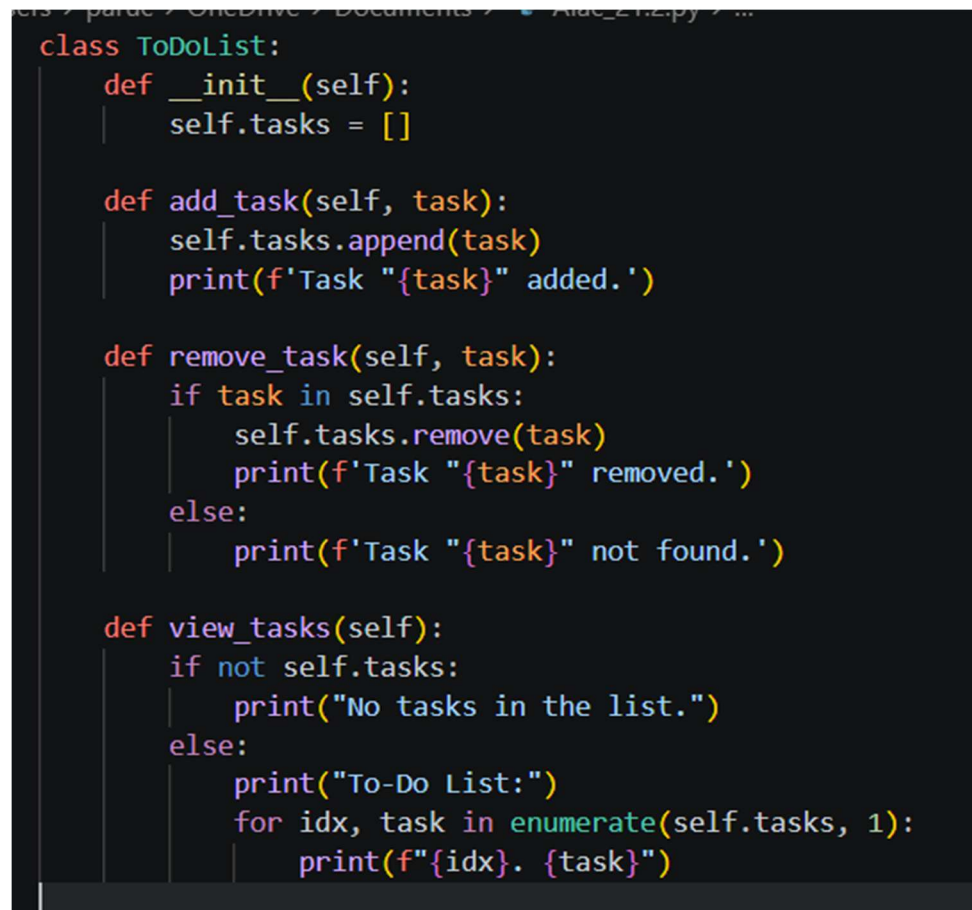
Ht.no: 2303A51434

Bt.no:21

Task 1 – AI-Assisted Pair Programming

- Use AI to generate an initial program (e.g., a simple To-Do List Manager).
- Ask AI to suggest enhancements (such as error handling, additional features, or refactoring).
- Compare the original and improved versions to understand the role of AI in collaboration.

Code screenshot:

A screenshot of a code editor showing a Python class named 'ToDoList'. The class has four methods: '__init__' which initializes a 'tasks' list; 'add_task' which appends a task and prints a confirmation; 'remove_task' which removes a task if it exists and prints a confirmation or an error message if not found; and 'view_tasks' which prints 'No tasks in the list.' if the list is empty, or 'To-Do List:' followed by a numbered list of tasks. The code is color-coded with syntax highlighting.

```
class ToDoList:
    def __init__(self):
        self.tasks = []

    def add_task(self, task):
        self.tasks.append(task)
        print(f'Task "{task}" added.')

    def remove_task(self, task):
        if task in self.tasks:
            self.tasks.remove(task)
            print(f'Task "{task}" removed.')
        else:
            print(f'Task "{task}" not found.')

    def view_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
        else:
            print("To-Do List:")
            for idx, task in enumerate(self.tasks, 1):
                print(f"{idx}. {task}")
```

Enhanced code screenshot:

```
# Enhanced Program with AI Suggestions
class ToDoList:
    def __init__(self):
        self.tasks = []

    def add_task(self, task):
        if task in self.tasks:
            print(f'Task "{task}" already exists.')
        else:
            self.tasks.append(task)
            print(f'Task "{task}" added.')

    def remove_task(self, task):
        if task in self.tasks:
            self.tasks.remove(task)
            print(f'Task "{task}" removed.')
        else:
            print(f'Task "{task}" not found.')

    def display_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
        else:
            print("To-Do List:")
            for idx, task in enumerate(self.tasks, start=1):
                print(f"{idx}. {task}")

    def clear_tasks(self):
        self.tasks.clear()
        print("All tasks cleared.")
```

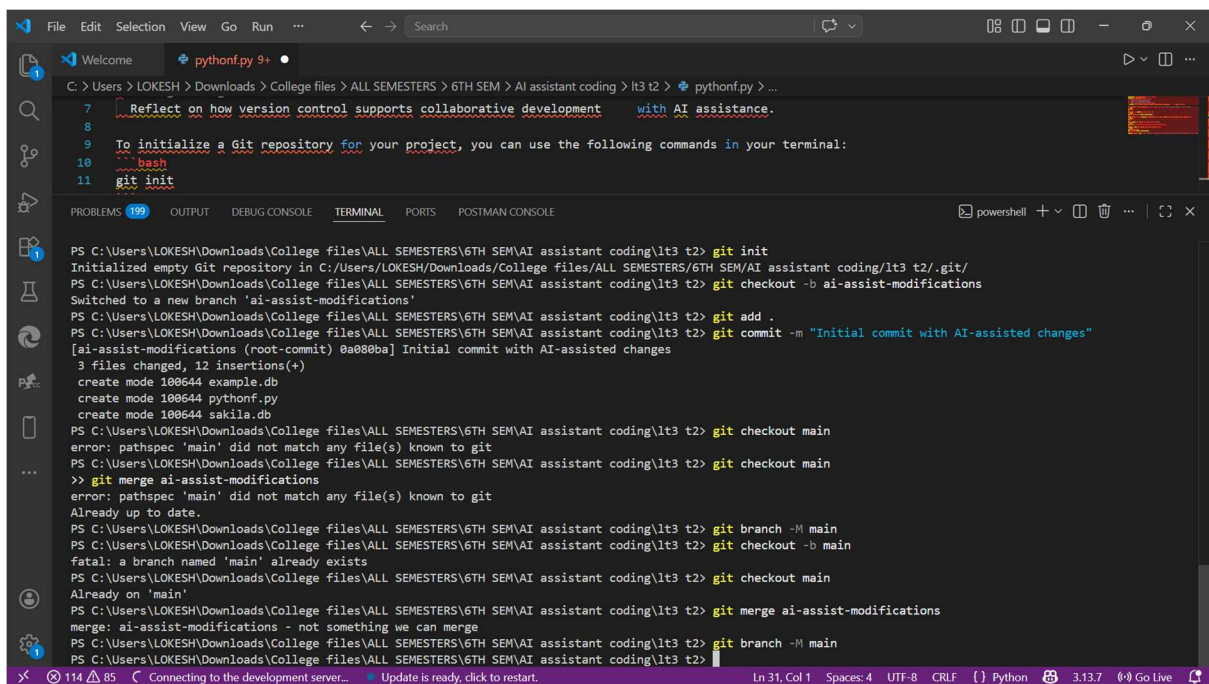
Task 2 – Version Control Integration

Initialize a Git repository for your project. Perform the following steps:

- Create and switch to a new branch.
- Add AI-assisted modifications.
- Commit with a meaningful message.
- Merge changes back into the main branch.

Reflect on how version control supports collaborative development with AI assistance.

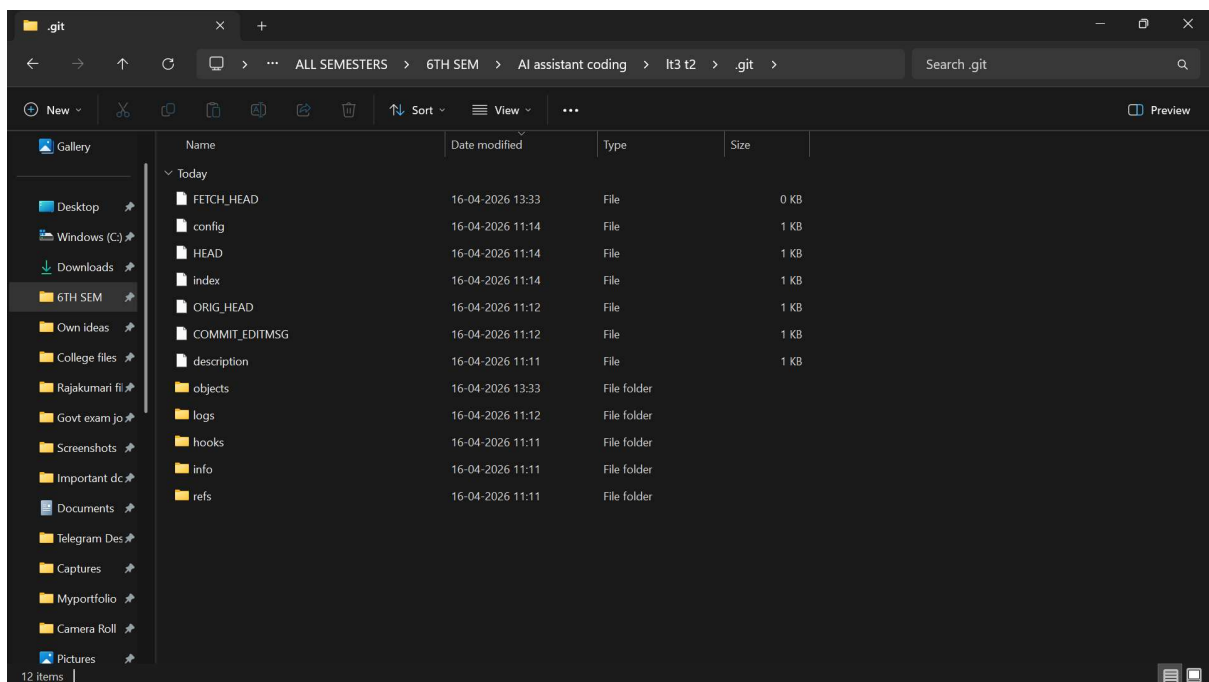
Code screenshot:



```
C:\Users\LOKESH> Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> pythonf.py ...
7 Reflect on how version control supports collaborative development with AI assistance.
8
9 To initialize a Git repository for your project, you can use the following commands in your terminal:
10 bash
11 git init

PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> git init
Initialized empty Git repository in C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2\.git\
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> git checkout -b ai-assist-modifications
Switched to a new branch 'ai-assist-modifications'
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> git add .
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> git commit -m "Initial commit with AI-assisted changes"
[ai-assist-modifications (root-commit) 0a080ba] Initial commit with AI-assisted changes
 3 files changed, 12 insertions(+)
 create mode 100644 example.db
 create mode 100644 pythonf.py
 create mode 100644 sakila.db
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> git checkout main
error: pathspec 'main' did not match any file(s) known to git
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> git checkout main
>> git merge ai-assist-modifications
error: pathspec 'main' did not match any file(s) known to git
Already up to date.
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> git branch -M main
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> git checkout -b main
fatal: a branch named 'main' already exists
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> git checkout main
Already on 'main'
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> git merge ai-assist-modifications
merge: ai-assist-modifications - not something we can merge
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2> git branch -M main
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\lt3 t2>
```

Output:



Task 3 – AI for Commit Message Generation

- Use AI to generate **concise, meaningful commit messages** based on the changes made.
- Compare AI-generated messages with human-written ones.
- Discuss how AI can improve **documentation and traceability** in Git history.

Code screenshot:

```
import os
import subprocess
def get_git_diff():
    """Get the git diff of the current repository."""
    try:
        result = subprocess.run(['git', 'diff', '--cached'], stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True)
        if result.returncode != 0:
            print("Error getting git diff:", result.stderr)
            return None
        return result.stdout
    except Exception as e:
        print("Exception occurred while getting git diff:", str(e))
        return None
def generate_commit_message(diff):
    """Generate a commit message based on the git diff."""
    if not diff:
        return "No changes to commit."

    # Simple heuristic for generating commit messages
    lines = diff.splitlines()
    added_lines = [line for line in lines if line.startswith('+') and not line.startswith('+++')]
    removed_lines = [line for line in lines if line.startswith('-') and not line.startswith('---')]

    if added_lines and removed_lines:
        return "Updated code with additions and deletions."
    elif added_lines:
        return "Added new code."
    elif removed_lines:
        return "Removed existing code."
    else:
        return "Modified code without clear additions or deletions."
def main():
    diff = get_git_diff()
    if diff is not None:
        commit_message = generate_commit_message(diff)
        print("AI-generated commit message:", commit_message)
if __name__ == "__main__":
    main()
```

Output:

```
ALL SEMESTERS/6TH SEM/AI assistant coding/22.5/Aiac_task2.py"
AI-generated commit message: No changes to commit.
```

Task 4 – Pair Programming Simulation

- Student A writes the initial version of a program.
- Student B, with AI assistance, suggests improvements or adds features.

Both students collaborate using Git branching and merging to integrate their work.

Code screenshot:

```

# Student A's initial version of the program
# A simple program to calculate the area of a rectangle
def calculate_area(length, width):
    return length * width
# Example usage
length = 5
width = 3
area = calculate_area(length, width)
print(f"The area of the rectangle is: {area}")
# Student B's improvements and feature additions
# Student B suggests adding input validation and a feature to calculate the perimeter
def calculate_area(length, width):
    if length <= 0 or width <= 0:
        raise ValueError("Length and width must be positive numbers.")
    return length * width
def calculate_perimeter(length, width):
    if length <= 0 or width <= 0:
        raise ValueError("Length and width must be positive numbers.")
    return 2 * (length + width)
# Example usage with improvements
length = 5
width = 3

try:
    area = calculate_area(length, width)
    perimeter = calculate_perimeter(length, width)
    print(f"The area of the rectangle is: {area}")
    print(f"The perimeter of the rectangle is: {perimeter}")
except ValueError as e:
    print(e)
# Git branching and merging
# Student A creates a branch for the initial version
# git checkout -b initial-version
# Student A commits the initial version
# git add Aiac_task2.py
# git commit -m "Initial version of the rectangle area calculator"
# Student B creates a branch for improvements
# git checkout -b improvements
# Student B commits the improvements
# git add Aiac_task2.py
# git commit -m "Added input validation and perimeter calculation"
# Student A merges the improvements into the main branch
# git checkout main
# git merge improvements
# The final version of the program now includes both the area and perimeter calculations with input validation

```

Git repository commands:

```

untracked files present (use "git add" to track)
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git checkout main
not match any file(s) known to git
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git checkout -b initial version
error: pathspec 'initial' cannot be created from it
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git init
Initialized empty Git repository in C:/Users/LOKESH/Downloads/College files/ALL SEMESTERS/6TH SEM/AI assistant coding/22.5/.git/
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git add Aiacc_task2.py
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git commit -m "Initial commit - rectangle area calculator"
[c] Initial commit - rectangle area calculator
ns(+)
k2.py
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git branch -M main
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git checkout -b initial-version
initial-version'
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git checkout main

\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git checkout -b improvements
improvements'
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git add Aiacc_task2.py
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git commit -m "Added validation and perimeter calculation"

o include in what will be committed)

untracked files present (use "git add" to track)
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git commit -m "Added validation and perimeter calculation"

o include in what will be committed)

untracked files present (use "git add" to track)
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git commit -m "Added validation and perimeter calculation"

ree clean
\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git checkout main

\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5> git merge improvements

\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5>

```